

Joseph Leone
4/4/2026
COP 3035C-042
Homework 5

Exercise 1

I created a program where the user enters calorie counts separated by commas and the program sums up the values and prints out all the entries with the total.

Part (a)

Homework5_ex1a.py

This is the version without list comprehensions.

```
# Homework 5
# Exercise 1a
# COP3035C
# Joseph Leone

def main():
    calorie_list = get_calorie_list()
    total = 0
    count = 1
    print("-"*64)
    for val in calorie_list:
        entry_str = f"Entry {count}"
        print(f"{entry_str:8}: {val:5}")
        count += 1
        total += val
    print(f"{'Total':8}: {total:5}")

def get_calorie_list():
    calorie_str = input("Input calories separated by a comma:\n")
    calorie_list = calorie_str.split(",")
    calorie_ints = []
    for val in calorie_list:
        calorie_ints.append(int(round(float(val), 0)))
    return calorie_ints

if __name__ == "__main__":
    main()
```

Part (b)

Homework5_ex1b.py

I used a list comprehension to get the integers list of all the calorie entries. It reduced the length of the program by two lines.

```
# Homework 5
# Exercise 1b
# COP3035C
# Joseph Leone

def main():
    calorie_list = get_calorie_list()
    total = 0
    count = 1
    print("-"*64)
    for val in calorie_list:
        entry_str = f"Entry {count}"
        print(f"{entry_str:8}: {val:5}")
        count += 1
        total += val
    print(f"'Total':8}: {total:5}")

def get_calorie_list():
    calorie_str = input("Input calories separated by a comma:\n")
    calorie_list = calorie_str.split(",")
    calorie_ints = [(int(round(float(val), 0))) for val in calorie_list]
    return calorie_ints

if __name__ == "__main__":
    main()
```

Example output

```
Input calories separated by a comma:
100, 200, 400, 500, 800
-----
Entry 1 :    100
Entry 2 :    200
Entry 3 :    400
Entry 4 :    500
Entry 5 :    800
Total   :   2000
```

Exercise 2, Part 1, homework5_ex2_part1.py

I created a menu so the user can select each operation. Each selection corresponds to a function call that handles the selection. There is a function that displays the menu and handles the user selection and another function that validates user input.

```
# Homework 5
# Exercise 2 Part 1
# COP3035C
# Joseph Leone

def main():
    library = {}
    while True:
        selection = get_selection()
        if selection == "1":
            add_book(library)
        elif selection == "2":
            find_author(library)
        elif selection == "3":
            update_author(library)
        elif selection == "4":
            remove_book(library)
        elif selection == "5":
            show_books(library)
        else:
            quit()

def remove_book(library):
    title = input("\nInput title: ").strip().lower()
    if title in library:
        library.pop(title)
        print(f"{title.title()} removed from library\n")
    else:
        print("Title not in library\n")

def update_author(library):
    title = input("\nInput title: ").strip().lower()
    author = input("Update author: ").strip().lower()
    if title in library:
        library.update({title: author})
        print("Author updated\n")
```

```

else:
    print("Title not in library\n")

def find_author(library):
    title = input("\nEnter title to get author: ").strip()
    author = library.get(title.lower())
    if author == None:
        print("Author not found\n")
    else:
        print(f"Author: {author.title()}\n")

def add_book(library):
    title = input("\nInput title: ").strip().lower()
    author = input("Author: ").strip().lower()
    if title in library:
        print("Book already in library\n")
    else:
        library[title] = author
        print("Book added\n")

def show_books(library):
    print("\nAll books")
    print("-"*64)
    for title, author in library.items():
        print(f"{title.title()} by {author.title()}")
    print()

def get_selection():
    while True:
        print(
            "1. Add book\n"
            "2. Find Author\n"
            "3. Update Author\n"
            "4. Remove book\n"
            "5. Show all books\n"
            "6. Quit"
        )
        selection = input("Input number: ").strip()
        if is_selection_valid(selection):
            return selection

```

```

        else:
            print("\n" + "*" * 64)
            print("Invalid selection")
            print("*" * 64)

def is_selection_valid(selection):
    valid_selections = ["1", "2", "3", "4", "5", "6"]
    if selection in valid_selections:
        return True
    else:
        return False

if __name__ == "__main__":
    main()

```

Example output

```

1. Add book
2. Find Author
3. Update Author
4. Remove book
5. Show all books
6. Quit
Input number: 1

Input title: the great gatsby
Author: f. scott fitzgerald
Book added

1. Add book
2. Find Author
3. Update Author
4. Remove book
5. Show all books
6. Quit
Input number: 5

All books
-----
The Great Gatsby by F. Scott Fitzgerald

```

Exercise 2, Part 2, homework5_ex2_part2.py

This Program creates two sets and does some set operations and prints to results to confirm.

```
# Homework 5
# Exercise 2 Part 2
# COP3035C
# Joseph Leone

common_categories = {"Fiction", "Non-Fiction", "Science", "History"}
categories = set()

# add categories
categories.add("Mystery")
categories.add("Romance")
categories.add("Science")
print(f"Common categories: {common_categories}")
print(f"Categories: {categories}", end="\n\n")

# remove category
categories.remove("Mystery")
print("'Mystery' removed")
print(f"Categories: {categories}", end="\n\n")

# intersection of categories and common categories
intersection = categories & common_categories
print("Intersection of categories and common categories")
print(intersection, end="\n\n")

# categories only in common_categories
difference = common_categories - categories
print("Elements in common categories but not in categories")
print(difference)
```

Output

```
Common categories: {'Science', 'Non-Fiction', 'Fiction', 'History'}
Categories: {'Science', 'Romance', 'Mystery'}

'Mystery' removed
Categories: {'Science', 'Romance'}

Intersection of categories and common categories
{'Science'}

Elements in common categories but not in categories
{'History', 'Non-Fiction', 'Fiction'}
```

Exercise 3, homework5_ex3.py

The user inputs food items along with their associated category. The program then outputs the shopping list with each item sorted by category. It also allows the user to confirm if one item is in the shopping list and shows its category.

```
# Homework 5
# Exercise 3
# COP3035C
# Joseph Leone

def main():
    shopping_list = get_shopping_list()
    show_shopping_list(shopping_list)
    find_item_category(shopping_list)

def show_shopping_list(shopping_list):
    print("\nShopping List")
    print("-"*64)
    for category, items in shopping_list.items():
        print(f"{category.capitalize()}: {[item.capitalize() for item in items]}")

def find_item_category(shopping_list):
    item = input("\nEnter item to find: ").strip().lower()
    found = False
    for category, item_list in shopping_list.items():
        if item in item_list:
            found = True
            print(f"{item.capitalize()} is in {category.capitalize()}")
    if found == False:
        print("Item not found")

def get_shopping_list():
    shopping_list = {}
    count = 1
    print("Add shopping list items")
    print("-"*64)
    while True:
        print("Add item or type 'Finish' to end")
        item = input(f"Enter item {count}: ").strip().lower()
        if item == "finish":
```

```

        break
    category = input(f"Enter category for item {count}:")
    category = category.strip().lower()
    if category not in shopping_list:
        shopping_list[category] = [item]
    else:
        shopping_list[category].append(item)
    count += 1
return shopping_list

if __name__ == "__main__":
    main()

```

Example output

```

Add shopping list items
=====
Add item or type 'Finish' to end
Enter item 1: milk
Enter category for item 1: dairy
Add item or type 'Finish' to end
Enter item 2: cheese
Enter category for item 2: dairy
Add item or type 'Finish' to end
Enter item 3: pork chops
Enter category for item 3: meat
Add item or type 'Finish' to end
Enter item 4: bacon
Enter category for item 4: meat
Add item or type 'Finish' to end
Enter item 5: lettuce
Enter category for item 5: vegetables
Add item or type 'Finish' to end
Enter item 6: finish

Shopping List
=====
Dairy: ['Milk', 'Cheese']
Meat: ['Pork chops', 'Bacon']
Vegetables: ['Lettuce']

Enter item to find: milk
Milk is in Dairy

```


Exercise 4

I decided to create a tic-tac-toe game. The first file is the one I made and the second is the one that Google Gemini gave me.

homework5_ex4.py

```
# Homework 5
# Exercise 4
# COP3035C
# Joseph Leone

def main():
    board = [
        [" ", " ", " "],
        [" ", " ", " "],
        [" ", " ", " "]
    ]
    player = "X"
    while True:
        print()
        take_turn(board, player)
        game_over_check(board, player)
        if player == "X":
            player = "O"
        else:
            player = "X"

def take_turn(board, player):
    while True:
        print(f"Player {player}'s turn")
        print("-"*64)
        show_board(board)
        row = input("Enter row: ").strip()
        cell = input("Enter cell: ").strip()
        if is_valid_move(board, row, cell) == False:
            print("\n" + "*" * 64)
            print("Invalid move")
            print("*" * 64)
            continue
        if player == "X":
            board[int(row)][int(cell)] = "X"
```

```

        else:
            board[int(row)][int(cell)] = "o"
        break

def game_over_check(board, player):
    diagonal_up = [board[2][0], board[1][1], board[0][2]]
    diagonal_down = [board[0][0], board[1][1], board[2][2]]
    vertical_0 = [board[0][0], board[1][0], board[2][0]]
    vertical_1 = [board[0][1], board[1][1], board[2][1]]
    vertical_2 = [board[0][2], board[1][2], board[2][2]]
    possible_wins = [
        diagonal_up, diagonal_down,
        vertical_0, vertical_1, vertical_2,
        board[0], board[1], board[2]
    ]
    for possible_win in possible_wins:
        if possible_win[0] != " " and len(set(possible_win)) == 1:
            print(f"\nPlayer {player} wins!")
            print("-"*64)
            show_board(board)
            exit()

    if " " not in [cell for row in board for cell in row]:
        print("Draw!")
        print("-"*64)
        show_board(board)
        exit()

def is_valid_move(board, row, cell):
    valid_nums = ["0", "1", "2"]
    if row not in valid_nums or cell not in valid_nums:
        return False
    if board[int(row)][int(cell)] != " ":
        return False
    return True

def show_board(board):
    print("Cells:   0    1    2")
    for index, row in enumerate(board):
        print(f"Row {index}: {row}")

```

```
if __name__ == "__main__":  
    main()
```

Homework5_ex4_ai.py

```
# Homework 5  
# Exercise 4 with AI  
# COP3035C  
# Joseph Leone  
# Prompt: Create a single file Python program for me. I want a tic-tac-toe  
game that uses the terminal to display the board and enter information.  
  
import os  
  
def clear_screen():  
    # Clears terminal screen based on OS  
    os.system('cls' if os.name == 'nt' else 'clear')  
  
def display_board(board):  
    print(f"\n {board[0]} | {board[1]} | {board[2]} ")  
    print("-----")  
    print(f" {board[3]} | {board[4]} | {board[5]} ")  
    print("-----")  
    print(f" {board[6]} | {board[7]} | {board[8]} \n")  
  
def check_win(board, player):  
    win_conditions = [  
        [0, 1, 2], [3, 4, 5], [6, 7, 8], # Horizontal  
        [0, 3, 6], [1, 4, 7], [2, 5, 8], # Vertical  
        [0, 4, 8], [2, 4, 6]             # Diagonal  
    ]  
    for condition in win_conditions:  
        if all(board[i] == player for i in condition):  
            return True  
    return False  
  
def check_draw(board):  
    return all(space in ['X', 'O'] for space in board)  
  
def play_game():  
    # Initialize board with numbers 1-9 for user guidance
```

```

board = [str(i + 1) for i in range(9)]
current_player = "X"
game_active = True

while game_active:
    clear_screen()
    print("--- TIC-TAC-TOE ---")
    display_board(board)

    print(f"Player {current_player}'s turn.")

    try:
        choice = int(input("Select a position (1-9): ")) - 1

        if choice < 0 or choice > 8 or board[choice] in ['X', 'O']:
            print("Invalid move! Try again.")
            input("Press Enter to continue...")
            continue

        board[choice] = current_player

        if check_win(board, current_player):
            clear_screen()
            display_board(board)
            print(f"Congratulations! Player {current_player} wins!")
            game_active = False
        elif check_draw(board):
            clear_screen()
            display_board(board)
            print("It's a draw!")
            game_active = False
        else:
            # Switch players
            current_player = "O" if current_player == "X" else "X"

    except ValueError:
        print("Please enter a valid number (1-9).")
        input("Press Enter to continue...")

if __name__ == "__main__":

```

```
play_game()
```

The biggest difference I see between my code and the code from the AI is that I have more code in the support functions and the AI has more code in the main function. I prefer the functionality of the program from the AI. It clears the screen so the board appears to be persistent when updating. Also the display of the board and method of selecting squares is simpler.

I think it is still important for people to learn to code still matters because if there is an issue someone needs to be able to have an understanding of the problem and the real world context of it.

Interaction with Gemini

Prompt:

Create a single file Python program for me. I want a tic-tac-toe game that uses the terminal to display the board and enter information.

Response:

Here is a complete, single-file Python program for a terminal-based Tic-Tac-Toe game. It includes a game loop, basic input validation, and a win-checking algorithm.

The Code

How to Run It

1. Ensure you have Python installed.
2. Copy the code above into a file named `tictactoe.py`.
3. Open your terminal or command prompt.
4. Run the command: `python tictactoe.py`

Key Features

- * **Dynamic Board:** The board initializes with numbers `**1-9**`, making it easy for players to see which key corresponds to which square.
- * **Error Handling:** If a player tries to pick a spot that is already taken or enters a letter instead of a number, the program gently asks them to try again.
- * **Cross-Platform:** The ``clear_screen`` function works on both Windows (``cls``) and macOS/Linux (``clear``), keeping the terminal tidy.
- * **Win Logic:** Uses a list of index combinations to check for three-in-a-row horizontally, vertically, and diagonally.